Chapitre 0 — Avant tout | Partie A — Écrire des lettres | leçon 0.7 sur 153 | difficulté 1 / 10

0.7. La boucle do … while : au moins un tour

La boucle qui vérifie sa condition à la fin : elle fait toujours au moins un tour.

1. Ce qu'il faut comprendre

`do { … } while (i < 20);` fait le tour **d'abord**, et vérifie la condition **ensuite**. Il fait donc **toujours au moins un tour**, même si la condition est fausse dès le départ.

Ici, le résultat est le même qu'avec `for` et `while` : les trois boucles savent faire la même chose. On choisit celle qui se lit le mieux.

Attention au **point-virgule** après `while (i < 20)` : il est obligatoire avec `do`, et seulement avec lui.

> **Les fonctions de cette leçon — et le tuto de chacune**
>
> - `#include <poser>` → 0.1.3. La fonction poser() — poser une tuile
> - `#include <ALPHABET>` → 0.1.4. ALPHABET — les lettres de la police

📖 Les mots de cette leçon — si tu débutes

- **accolades { }** — Elles entourent un bloc : les lignes qui vont ensemble (le corps d'une fonction, d'une boucle, d'un « if »). Chaque « { » a son « } ».
- **affectation (=)** — « x = 5; » : la boîte x prend la valeur 5. Ce n'est pas une égalité : c'est un rangement.
- **argument** — Une valeur donnée à une fonction quand on l'appelle, entre ses parenthèses : dans texte(5, 8, "SALUT"), 5, 8 et "SALUT" sont les arguments.
- **bloc** — Un groupe de lignes entre accolades { }, qui s'exécutent ensemble.
- **booléen (vrai / faux)** — Le résultat d'une condition : vrai (1) ou faux (0). « true » veut dire vrai, « false » faux.
- **boucle** — Des instructions qui se répètent : « while » (tant que…) ou « for » (pour un compteur qui va de… à…).
- **boucle du jeu** — « while (true) { image(); … } » : ce qui se répète 60 fois par seconde, sans fin, pendant tout le jeu.
- **commentaire (//)** — Tout ce qui suit « // » jusqu'au bout de la ligne : une phrase pour l'humain, que la console ignore.
- **condition** — Une question dont la réponse est vrai ou faux : « score > 10 », « bouton(A) ». On la teste avec « if » ou « while ».
- **coordonnées** — Deux nombres qui disent où : la colonne (vers la droite) puis la ligne (vers le bas), à partir de (0, 0) en haut à gauche.
- **image()** — Attend l'image suivante. La console en montre 60 par seconde : c'est ce qui règle la vitesse d'un jeu.
- **#include** — En haut du programme : « #include <texte> » demande la fonction texte() à la console ; « #include "fichier.cpp" » verse un de tes fichiers à cet endroit.
- **incrémenter (++)** — « x++ » ou « x += 1 » : x augmente de 1. « x-- » : x diminue de 1.
- **indice** — Le numéro d'une case dans un tableau, entre crochets : cases[3] est la quatrième case (on compte à partir de 0).
- **instruction** — Un ordre donné à la console, terminé par un point-virgule : « texte(1, 1, "OK"); ».
- **main** — « int main() { … } » : là où le programme commence. La console exécute d'abord la première ligne de main.
- **octet** — Un nombre de 0 à 255 : la plus petite boîte de la mémoire. Une cartouche en a 32 768.
- **opérateurs de comparaison** — == égal, != différent, < plus petit, > plus grand, <= plus petit ou égal, >= plus grand ou égal. Le résultat est vrai ou faux.
- **point-virgule (;)** — Il termine une instruction, comme le point termine une phrase.
- **poser()** — Met une tuile dans le décor, sur une case de la grille de 20 × 18 (32 × 32 en tout, avec la partie cachée).
- **uint8_t** — Le genre de nombre de cette console : un entier de 0 à 255 (un octet). Au-delà, il repart à 0.
- **variable** — Une boîte avec un nom, dans la mémoire, qui retient un nombre qu'on peut lire et changer.
- **while (tant que)** — Répète un bloc tant qu'une condition est vraie.

🔍 Le programme, ligne par ligne — chaque ligne dite en français

`#include <poser>  // pose une tuile sur une case du fond`	Demande à la console « poser » (ce que c'est : pose une tuile sur une case du fond). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « pose une tuile sur une case du fond ».)

`#include <ALPHABET>  // les lettres de la police : ALPHABET[0] est le A`	Demande à la console « ALPHABET » (ce que c'est : les lettres de la police : ALPHABET[0] est le A). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « les lettres de la police : ALPHABET[0] est le A ».)
`int main() {  // Le programme commence ici.`	Le début du programme : la console commence ici, par la première ligne qui suit. (Le commentaire le dit : « Le programme commence ici. ».)
`uint8_t i = 0;  // Une variable i : le numéro de la lettre ET de la colonne.`	Une variable : une boîte appelée i, qui retient un nombre de 0 à 255, et qui commence à 0. (Le commentaire le dit : « Une variable i : le numéro de la lettre ET de la colonne. ».)
`// uint8_t = un nombre entier de 0 à 255. On part de 0.`	Un commentaire : une phrase pour celui qui lit (« uint8_t = un nombre entier de 0 à 255. On part de 0. »). La console l'ignore.
`do {  // do = « fais » : le bloc est fait AU MOINS une fois...`	Une boucle « fais… tant que » : on fait le bloc une première fois, puis on recommence tant que la condition écrite à la fin est vraie. (Le commentaire le dit : « do = « fais » : le bloc est fait AU MOINS une fois... ».)
`poser(i, 0, ALPHABET[i]);  // la lettre n° i, dans la colonne i`	Appel de la fonction poser de la console, avec les arguments i, 0, ALPHABET[i]. Ce qu'elle fait : pose une tuile sur la grille du fond — un nom de Tuile, ou un numéro. Un 4e argument est une TAILLE : poser(2, 3, MUR, 5) dessine MUR cinq fois plus grand dans le fond (colonne et ligne en clair). (Le commentaire le dit : « la lettre n° i, dans la colonne i ».)
`// (ALPHABET[0] = A, ALPHABET[1] = B, ...)`	Un commentaire : une phrase pour celui qui lit (« (ALPHABET[0] = A, ALPHABET[1] = B, ...) »). La console l'ignore.
`i++;  // i gagne 1 : la colonne et la lettre suivantes.`	i augmente de 1. (Le commentaire le dit : « i gagne 1 : la colonne et la lettre suivantes. ».)
`} while (i < 20);  // la condition est vérifiée À LA FIN :`	La fin de la boucle « do » : on recommence tant que i est plus petit que 20. (Le commentaire le dit : « la condition est vérifiée À LA FIN : ».)
`// ... puis recommencé tant que i < 20.`	Un commentaire : une phrase pour celui qui lit (« ... puis recommencé tant que i < 20. »). La console l'ignore.
`// Quand i vaut 20 (après le T), on sort.`	Un commentaire : une phrase pour celui qui lit (« Quand i vaut 20 (après le T), on sort. »). La console l'ignore.
`while (true) {  // La boucle de jeu : recommence pour toujours...`	La boucle du jeu : tout ce qui est entre les accolades recommence sans fin, 60 fois par seconde avec image(). (Le commentaire le dit : « La boucle de jeu : recommence pour toujours... ».)
`image();  // ... attendre l'image suivante.`	Attendre l'image suivante : la console en montre 60 par seconde. C'est ce qui règle la vitesse du jeu. (Le commentaire le dit : « ... attendre l'image suivante. ».)
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **887 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
  démarrage        49 o  ← la console, à l'allumage  — la mise en route de la console
  main()           40 o  ← démarrage  — ta fonction
  VBlank            8 o  ← la console, à chaque image  — compte les images, 60 fois par seconde
  AttendreImage    28 o  ← image()  — attend l'image suivante
  AttendreVBlank   16 o  ← démarrage  — attend que l'écran soit libre
  AdresseCase      10 o  ← poser()  — trouve l'adresse d'une case de l'écran
  CopierTuiles     18 o  ← démarrage  — copie les lettres et les dessins en mémoire vidéo
  EffacerCarte     14 o  ← démarrage  — vide les deux cartes de l'écran
  Tuiles          704 o  ← CopierTuiles  — la police entière
Lettres gravées : toutes (poser() reçoit un numéro calculé)
Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, EffacerCases, EcrireTexte, EcrireNombre,
Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond,
EffacerPanneau, LireManette, DonneesTransfert, Notes
Fonctions du langage appelées :
  poser() × 1    18 o sur place, dans main()  + AdresseCase (10 o)
  image() × 1     3 o sur place, dans main()  + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <poser>      // pose une tuile sur une case du fond
#include <ALPHABET>   // les lettres de la police : ALPHABET[0] est le A
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                    // Le programme commence ici.
  uint8_t i = 0;                // Une variable i : le numéro de la lettre ET de la colonne.
                                // uint8_t = un nombre entier de 0 à 255. On part de 0.

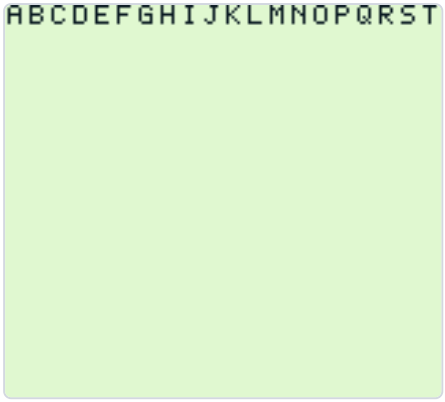
  do {                          // do = « fais » : le bloc est fait AU MOINS une fois...
    poser(i, 0, ALPHABET[i]);   // la lettre n° i, dans la colonne i
                                // (ALPHABET[0] = A, ALPHABET[1] = B, ...)
    i++;                        // i gagne 1 : la colonne et la lettre suivantes.
  } while (i < 20);             // la condition est vérifiée À LA FIN :
                                // ... puis recommencé tant que i < 20.
                                // Quand i vaut 20 (après le T), on sort.

  while (true) {                // La boucle de jeu : recommence pour toujours...
    image();                    // ... attendre l'image suivante.
  }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.

la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

> A à T sur toute la première ligne, comme au 0.5.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la ligne 0 est pleine : A à T
- ✓ rien ne déborde sur la ligne 1

Ensuite : **0.7.1. La boucle do … while : au moins un tour — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++